The SGA 3D Graph

Where the data comes from, how it is transformed, and how the graph is built

Covers the Box-document pipeline in `SGA-3D-Graph-Ready (1)/` and the live `#graph` page on branch `feat/graph-3d-live`. Figures in this document come from a fresh run of the build scripts on the supplied `Archive.zip`.

1. The short answer 2
2. Pipeline A at a glance 3
3. The source: Box documents 4
   What a Box contains 4
4. Step 1: parse_boxes.py (Word → boxes.json) 5
   How it works 5
   Key functions 5
   Worked example: Explore Box 07 5
5. Step 2: build_graph.py (inference) 7
   5.1 Event kind of each element 7
   5.2 Transitions (screen → screen links) 7
   5.3 Reuses 7
   5.4 Foundations (which shared service a dataset comes from) 8
   5.5 Output: page-data.json 8
6. Step 3: make_rev2.py (build the graph) 9
   6.1 What becomes a node 9
   6.2 How the dataset decides Db or API 9
   6.3 What becomes a link 10
   6.4 Hard-coded mappings to know about 10
   6.5 Output 10
7. Step 4: drawing it (template_rev2.html) 11
   Other pages built from the same data 11
8. Pipeline B: the live #graph page 12
   8.1 Where the data comes from 12
   8.2 How it is transformed: graphview.Build 12
   8.3 Endpoint and page 13
   8.4 Layers in the live view 13
9. What is real and what is inferred (pipeline A) 14
10. File reference 15
   Rebuild pipeline A after the Boxes change 15
   See pipeline B 15

1. The short answer

There are **two different 3D graphs**, built by two independent pipelines. They look alike but read completely different sources. Knowing which one you are showing is the first thing to get right.

	A. Box-document graph (rev 2)	B. Live app graph (#graph)
What it shows	The product as **specified**: every screen described in the Box Word documents, its UI elements, rules and data	The product as **running**: the State graph the backend actually serves
Source of data	49 Box `.docx` files in `Archive.zip`	`infra/seed/graph.yaml` loaded into Postgres (or Neo4j)
Transformed by	Python: `parse_boxes.py` → `build_graph.py` → `make_rev2.py`	Go: `graphview.Build` (runs the Spider on every State)
Delivered as	One self-contained HTML file: `sga-graph-rev2.html`	JSON from `GET /api/graph/architecture`
Drawn by	`template_rev2.html` + 3d-force-graph from a CDN	React page `frontend/src/pages/GraphView/index.tsx` + react-force-graph-3d
When it updates	Only when someone re-runs the scripts	Every page load, or "Reload from the graph"
Size today	3,595 nodes, 4,205 links	Depends on the seeded graph

The file you have open, `build_graph.py`, is the middle step of pipeline A. Sections 2 to 7 explain pipeline A end to end; section 8 explains pipeline B.

> One-line pitch for pipeline A: "We take the Word documents that specify each screen, parse their tables into JSON, infer how screens link to each other and which services they use, then lay all of it out as a four-layer 3D graph in a single HTML page."

2. Pipeline A at a glance

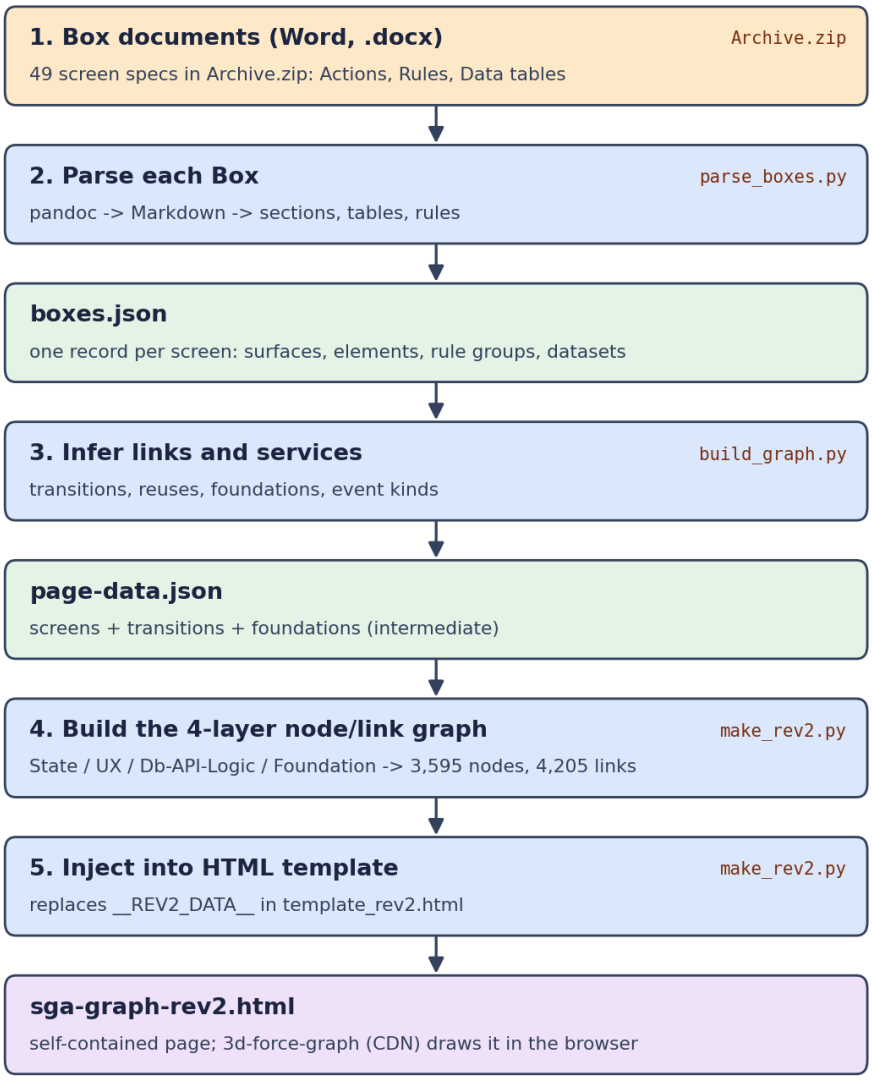


Figure 1. From Box documents to the 3D page. Blue = code that transforms; green = data files it produces.

Step	Input → output	Command
Parse	`Archive.zip` → `boxes.json` (944 KB)	`python3 parse_boxes.py Archive.zip boxes.json`
Infer + build + inject	`boxes.json` + `template_rev2.html` → `page-data.json` + `sga-graph-rev2.html` (1.4 MB)	`python3 make_rev2.py boxes.json template_rev2.html sga-graph-rev2.html`

`make_rev2.py` imports `build_graph.py` and calls its `main()` first, so `build_graph.py` never needs to be run by hand. `page-data.json` is written next to the output HTML as a side effect.

Requirements: Python 3 and pandoc (for step 1 only). Opening the finished HTML needs internet access because the 3D library loads from a CDN.

3. The source: Box documents

A **Box** is a Word document that specifies one screen (one "state") of the product. They live in Google Drive in an `Archive` folder, one sub-folder per module, and are downloaded as `Archive.zip`.

Module	Boxes parsed	Share
Value	20	41%
Build	15	31%
Explore	9	18%
Plan	4	8%
Command	1	2%
Total	**49**	

Only files whose name starts with `Box ` are read. Anything else in the zip (for example `Command Audit Template.docx`) is skipped.

What a Box contains

Every Box follows the same template. The parser looks for these headings:

Heading in the Word doc	What it holds	Becomes in the graph
Title (first line)	"Box 07 — Cooper Recommendations (Explore)"	The state node's number, name and id
Screen description	Prose about the screen	State description
The Actions	Tables of UI elements, grouped under surface headings. Columns: **Element / What it does / Type / Mock** (Figma link)	Surfaces and UI elements (UX layer); transitions
The Rules	Rule-group headings with numbered rules	Logic subgraph
The Data	Table with **Dataset / Columns / Source**	Db and API subgraphs, foundation services

4. Step 1: parse_boxes.py (Word → boxes.json)

Purpose: turn each Word document into one structured JSON record. Nothing is inferred here; it is a faithful extraction of what the Box says.

How it works

1. **Open the zip and pick Boxes.** `main()` loops over every `.docx` whose path contains `/Box` . The module name comes from the folder: `Explore Boxes` → `Explore`.
2. **Convert Word to Markdown.** Each file is run through `pandoc <file> -t gfm --wrap=none`. Word tables become Markdown pipe tables, which are easy to parse line by line.
3. **Walk the Markdown.** `parse_markdown()` reads line by line with a small state machine. When a line matches a section name in `SECTION_NAMES` it switches `section` to `description`, `actions`, `rules` or `data`.
4. **Read tables.** A run of lines starting with `|` is a table. `parse_table()` splits it into rows keyed by lower-cased header. In **The Actions** a table with an "Element" column gives UI elements; in **The Data** a table with a "Dataset" column gives datasets.
5. **Read headings and rules.** In The Actions, a short line without a full stop (≤ 90 chars) starts a new **surface** (a part of the screen, e.g. "Explore header"). In The Rules, a short line or bold heading starts a **rule group**; numbered lines become rules in it.
6. **Pick up Figma links.** The `FIGMA` regex pulls the file URL and `node-id` from the Mock column. The most common file URL becomes the Box's `figma` link; each element keeps its own node id.
7. **Name and id the Box.** `box_number_and_name()` splits the title into number and name ("07", "Cooper Recommendations"). The id is `<module>-<number>`, e.g. `explore-07`.
8. **Sort and write.** Boxes are sorted by module and number and written to `boxes.json`.

Key functions

Function	Role
`clean(text)`	Strips Markdown bold, links, HTML spans and escapes to plain text
`split_row` / `parse_table`	Turn a Markdown pipe table into a list of {header: cell} dicts
`col(row, *names)`	Finds a column by header prefix, so "What it does" and "What" both match
`parse_markdown(md, module, file)`	The state machine that fills `surfaces`, `ruleGroups`, `datasets`, `notes`, `figma`
`box_number_and_name(title, stem)`	Parses "Box 02a — ... (Explore)" into ("02a", name), dropping the module suffix
`main(zip, out)`	Unzips, calls pandoc, parses, sorts, writes JSON

Worked example: Explore Box 07

One row of the Actions table under the "Explore header" surface in the Word document:

Element	What it does	Type	Mock
"7 risks surfaced" pill	Opens the Cooper panel on the recommendation list. Hidden while Cooper has surfaced nothing.	Local change	Figma link, node 29:351488

Becomes this in `boxes.json`:

5. Step 2: build_graph.py (inference)

Purpose: add the things a single Box cannot tell you on its own: **how screens connect** and **which shared services they rely on**. This is the only step that interprets text rather than copying it, so it is where to be careful when presenting.

It is called from `make_rev2.py`, reads `boxes.json` and writes `page-data.json`.

5.1 Event kind of each element

Only elements of Type **Navigation**, **Local change** or **External** are treated as interactive. `event_kind()` labels each one from its name and text:

- `text_entry` if the label looks like an input (`TEXT_ENTRY` regex: input, search field, editor, "ask ...")
- `menu_choice` if it mentions a menu, dropdown, tab, filter, chip, toggle, select ... (`MENU` regex)
- `button_click` otherwise

5.2 Transitions (screen → screen links)

A transition says "an element on screen A takes you to screen B". They are found in four ways, strongest first. When the same pair is found more than once, the strongest basis is kept.

Basis	Rule in the code	Example	Count
`box-ref`	The element's "What it does" text names another Box: regex `Box\s+(\d+...)`, resolved by `resolve_box_ref()` (prefers the same module, or the module named in brackets)	"... specified in Box 14"	46
`module-nav`	A Navigation element goes to a module or home ("Goes to Value", the top nav, "Explore full map"). Target is the module's landing screen from `LANDING`	"Goes to Value" → `value-01`	13
`name-match`	The element text contains a known alias of another screen **in the same module** (the hand-written `ALIASES` table). Longest alias wins	"Goes to the investment details" → `build-06`	26
`demo-path`	Hard-coded in `DEMO_LINKS` to match the demo video click path	`explore-01` → `explore-07`	1*
		Total	**86 found, 85 drawn**

* `DEMO_LINKS` holds two links; the other one was already found by a stronger rule, so it keeps that basis. One `module-nav` link (Explore → Results) is dropped when drawing, because the Results module has no Box.

> The code itself says name-match links are "a first pass for people to review, not ground truth". 26 of the 85 transitions come from that alias table.

5.3 Reuses

Notes such as "As Box 00 — Shared Components ..." mean a screen reuses another Box's parts. These become 6 `reuses` records. They are kept in `page-data.json` but are **not drawn** by `make_rev2.py`.

5.4 Foundations (which shared service a dataset comes from)

The Data tables contain free-text "Source" phrases (over 300 different ones). `foundation_for_source()` maps each phrase to one shared service with the ordered keyword list `FOUNDATIONS`; the **first match wins**, and `other` catches the rest. External actions are mapped the same way with the `EXTERNAL` list (`external_for()`).

Foundation	Kind	Keywords (abridged)	Uses
Cooper AI service	service	cooper	128
Plan and portfolio records	database	plan, scenario, portfolio	55
Other or unnamed source	database	(anything else)	43
Investment records	database	investment, cost, budget	42
Value and strategy frameworks	database	framework, goal, formula	37
Asset register and risk models	database	asset, risk, sensor	34
Collaboration store	database	comment, thread, presence	29
Voice input service	service	voice, mic (external actions)	26
Document store	database	document, file, photo	20
GIS and map service	service	gis, map, tile, feeder	18
User directory	database	user, role, team	18
8 smaller ones		files, search, versions, session, notify, other external	46

5.5 Output: page-data.json

6. Step 3: make_rev2.py (build the graph)

Purpose: turn `page-data.json` into the final **nodes and links** in the four-layer SGA structure, then paste them into the HTML template.

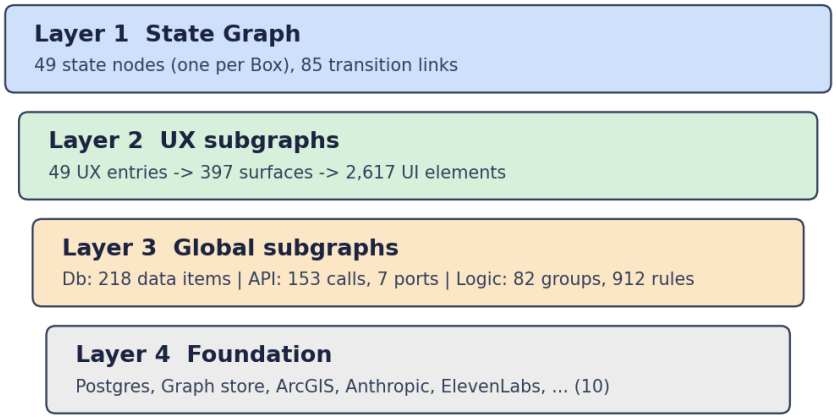


Figure 2. The four layers of the rev 2 graph, with today's counts.

6.1 What becomes a node

Layer	Node kind	Made from	Id pattern
1 State	`state`	One per Box	`explore-07`
2 UX	`ux` (entry)	One per state	`ux:explore-07`
2 UX	`surface`	Each surface heading in The Actions	`sf:explore-07:0`
2 UX	`element`	Each row of an Actions table	`el:explore-07:0:0`
3 Global	`entry`	One each for Db, API, Logic (whole app)	`db:entry`
3 Db	`group` / `item`	Data area / one dataset, de-duplicated by area + name	`area:assets`, `data:12`
3 API	`group` / `item`	Port / one outside call (External elements and service datasets)	`port:cooper`, `call:5`
3 Logic	`item`	One per rule group of a Box	`logic:explore-07:0`
4 Foundation	`foundation`	Fixed list `FOUNDATION`, only those in use	`fd:postgres`, `fd:anthropic`

Shared, not copied. A dataset or API call used by several screens is one node, with a `users` list of the states that use it. This is done by keying on `(area, norm(name))` or `(port, norm(name))`, where `norm()` lower-cases and strips punctuation.

6.2 How the dataset decides Db or API

For each dataset, the foundation chosen in step 2 decides the side: a `service` foundation (Cooper, GIS, voice, search, ...) becomes an **API call** under a port; a `database` foundation becomes a **Db item** under a data area.

6.3 What becomes a link

Link kind	From → to	Meaning
`has_ux`	state → ux	Each state owns exactly one UX subgraph
`has_surface`, `has_element`	ux → surface → element	Screen structure
`uses_data`	ux → Db item	This screen reads this dataset
`uses_api`	element or ux → API call	This element or screen calls an outside service
`uses_logic`	ux → Logic item	This screen follows these rules
`contains`	entry → group → item	Hierarchy inside Db, API and Logic
`resolves_to`	Db entry / port → foundation	Where it really runs (Postgres, ArcGIS, Anthropic ...)
`planned`	logic:entry → fd:calc	Calculation service, not built yet
`transition`	state → state	From step 2, with `basis` and `via`

6.4 Hard-coded mappings to know about

- `COMPONENTS`: UI component type (button, tab, dropdown ...) is **estimated from the element name**, because Boxes do not record it. 1,216 of 2,617 elements fall into "other".
- `DB_AREAS` and `API_PORTS`: fixed names for the groups in Layer 3.
- `FOUNDATION`: the real systems (Postgres, Neo4j, ArcGIS, Anthropic, ElevenLabs ...). These names are chosen in the script, not read from the Boxes.

6.5 Output

The script builds `{meta, nodes, links}`, serialises it as JSON (escaping `</` so it cannot close the script tag), and replaces the placeholder `__REV2_DATA__` inside `template_rev2.html`:

meta field	Value today
states / surfaces / elements	49 / 397 / 2,617
Db items / areas	218 / 10
API calls / ports	153 / 7
Logic groups / rules	82 / 912
Foundation nodes	10
Transitions	85
Total	3,595 nodes, 4,205 links, 1.4 MB page

7. Step 4: drawing it (template_rev2.html)

The finished page needs no server. When it opens in a browser:

1. It reads the embedded JSON: `JSON.parse(document.getElementById('rev2-data').textContent)`.
2. **Each node is pinned to its layer height.** It sets `n.fy` (fixed y) from the node kind: state, ux, surface, element, entry, group, item, foundation each have their own height. That is why the graph reads as floors.
3. **States are grouped by module.** Each state starts near its module's position (`MODULE_XZ`), and its UX nodes start under it. Db, API and Logic sit in their own regions (`REGION_X`). Foundations are fully fixed (`fx`, `fz`).
4. **The force layout arranges nodes within each floor.** 3d-force-graph (loaded from a CDN, on three.js) runs a physics simulation; link distance and charge strength are tuned per kind, and the default centring force is switched off.
5. It builds lookup maps for the side panel (children, users of each shared item, links per node) and draws the controls: layer toggles, search, "find all dropdowns", the details panel with Figma links, and a guided tour.

Other pages built from the same data

These do not re-parse the Boxes. They copy the JSON out of `sga-graph-rev2.html` into another template.

Page	Built by	Notes
`sga-vision.html`	`SGA-Vision-update/make_vision.py` + `template_vision.html`	Apple Vision Pro version
`sga-spatial.html`	`make_vision.py` + `template_spatial.html`	Adds glow, live pulses, a 10-step tour
`sga-atlas.html`	`SGA-Atlas/make_atlas.py` + `template_atlas.html`	Atlas view of the same states and subgraphs

> Two copies of make_rev2.py exist. The one in `SGA-Vision-update/` has a stricter "dropdown list box" rule (27 dropdowns instead of 49). The figures in this document come from the copy in `SGA-3D-Graph-Ready (1)/`.

8. Pipeline B: the live #graph page

This is the in-app version, added on branch `feat/graph-3d-live` (not committed yet). Instead of Word documents it reads the **real graph the backend serves**, so it can never drift from the running app.

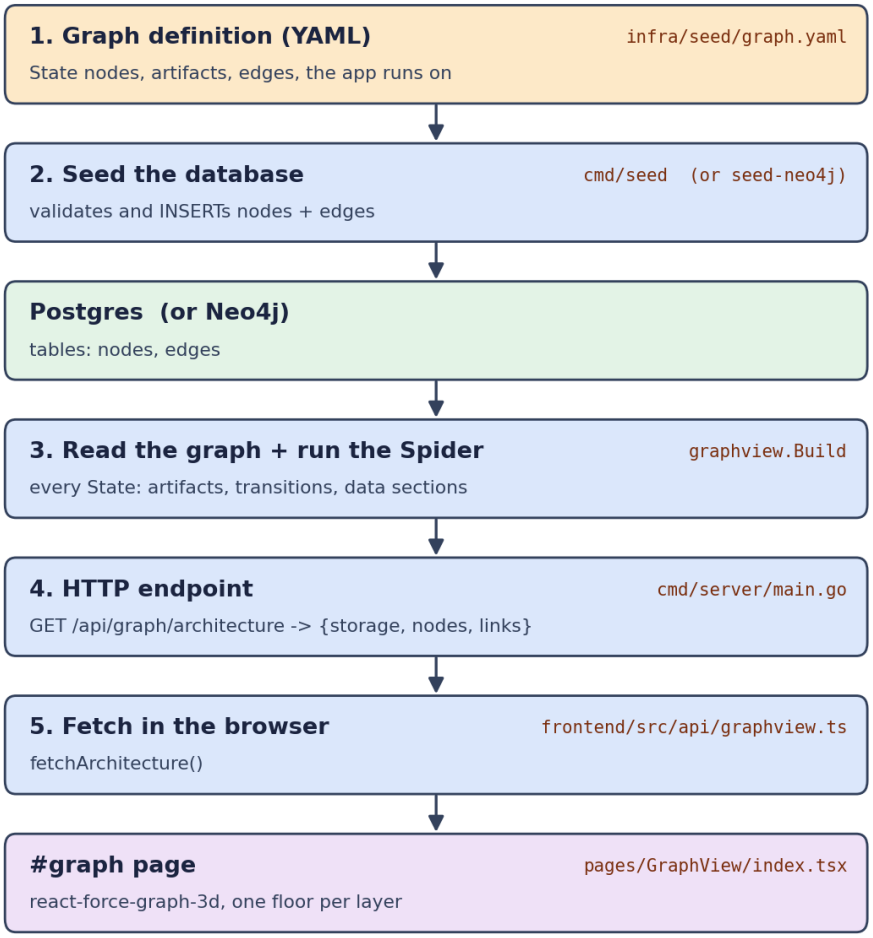


Figure 3. From the seed YAML to the #graph page.

8.1 Where the data comes from

- `infra/seed/graph.yaml` (plus `asset-details.yaml`) declares nodes (State, artifacts, data) and edges.
- `backend/cmd/seed/main.go` validates the files and runs `INSERT INTO nodes ...` and `INSERT INTO edges ...` in Postgres. `cmd/seed-neo4j` does the same for Neo4j.
- The server picks the store from `GRAPH_BACKEND` (`postgres` by default, or `neo4j`) and reads it through the `graph.GraphReader` interface (`backend/internal/graph/reader.go`).

8.2 How it is transformed: graphview.Build

File: `backend/internal/graphview/graphview.go`.

1. `reader.GetNodesByType(ctx, "State")` lists every State, sorted by id.
2. Adds a `spider` node and a `foundation:<storage>` node.

3. **Runs the Spider on every State** (8 in parallel via `errgroup`). The Spider result says which data items each screen section holds. A Spider error is stored on the State as `spiderError` rather than failing the whole view.
4. For each State: adds the State node, a `serves` link from the Spider, and `state_transition` links from `reader.GetEdges(...)`.
5. Reads three artifacts per State with `reader.GetArtifact`: **UX**, **DATA_COLLECT**, **BUSINESS_RULE**. Under UX, each `viewHints` key becomes a UX node. Under DATA_COLLECT, a `collects` link goes to each data type the Spider found. Under BUSINESS_RULE, `rules()` splits the spec into rules, and each rule gets `reads` links to the data types of the sections it names.
6. Finally one `datatype:<Type>` node per data type seen, with `nodesCollected`, linked `stored_in` the foundation.

8.3 Endpoint and page

File	Role
`backend/cmd/server/main.go`	Registers `GET /api/graph/architecture`, calls `graphview.Build` and returns `{storage, nodes, links}`
`frontend/src/api/graphview.ts`	`fetchArchitecture()` and the `ViewNode` / `ViewLink` types
`frontend/src/main.tsx`	When the URL hash is `#graph`, lazy-loads `GraphView` behind the `AuthGate` (three.js stays out of the main bundle)
`frontend/src/pages/GraphView/index.tsx`	Pins each node to its layer's height (`fy` from `LAYERS`), pulls Data and Logic sideways with `forceX`, draws with react-force-graph-3d. Search, "find all UX of a kind", subtree highlight, layer toggles
`backend/internal/graphview/graphview_test.go`	Tests for the builder
`e2e/graph-view.spec.ts`	End-to-end test of the page

8.4 Layers in the live view

Layer	Contents
Spider	One node: the engine that serves every State
State	Every State node, with `state_transition` arrows
Artifact	UX, DATA_COLLECT and BUSINESS_RULE of each State
UX	One node per `viewHints` key
Logic	One node per business rule
Data	One node per data type the Spider collected, sized by how many
API	Empty: the graph has no API artifact yet
Foundation	The storage backend (postgres or neo4j)

9. What is real and what is inferred (pipeline A)

Useful when someone asks "is this link in the spec?". Ordered from most to least reliable.

Part of the graph	Reliability	Why
States, surfaces, elements, rules, datasets	From the Box	Copied from the Word tables and headings by parse_boxes.py
Figma links on elements	From the Box	Taken from the Mock column
`box-ref` transitions (46)	Strong	The Box text names the target Box
`module-nav` transitions (13, 12 drawn)	Strong	Explicit "Goes to <module>" or top navigation
Foundation of each dataset	Inferred	First keyword match on the free-text Source; 43 fall into "other"
`name-match` transitions (26)	Inferred	Hand-written alias list; the code says to review them
Component type (button, dropdown ...)	Estimated	Guessed from the element name; 1,216 are "other"
Foundation systems (Postgres, ArcGIS, Anthropic ...)	Chosen	Fixed in `make_rev2.py`, not in the Boxes
`demo-path` link	Hand-added	Matches the demo video

10. File reference

File	What it does
`SGA-3D-Graph-Ready (1)/Archive.zip`	Source: the Box Word documents
`SGA-3D-Graph-Ready (1)/parse_boxes.py`	Word → `boxes.json` (extraction)
`SGA-3D-Graph-Ready (1)/build_graph.py`	`boxes.json` → `page-data.json` (transitions, foundations, event kinds)
`SGA-3D-Graph-Ready (1)/make_rev2.py`	Builds nodes and links, injects into the template
`SGA-3D-Graph-Ready (1)/template_rev2.html`	Renderer: layout, controls, details panel
`SGA-3D-Graph-Ready (1)/sga-graph-rev2.html`	Generated page to open (not the template)
`SGA-Vision-update/make_vision.py`	Vision Pro and spatial pages from rev 2 data
`SGA-Atlas/make_atlas.py`	Atlas page from rev 2 data
`infra/seed/graph.yaml`	Source of the live graph
`backend/cmd/seed/main.go`	Loads the YAML into Postgres
`backend/internal/graphview/graphview.go`	Builds the live layered view
`backend/cmd/server/main.go`	`GET /api/graph/architecture`
`frontend/src/pages/GraphView/index.tsx`	The `#graph` 3D page

Rebuild pipeline A after the Boxes change

See pipeline B